

Chapitre 6:

SQL: Langage d'interrogation des données

Objectifs du cours.

1. Définir le LID
2. Savoir formuler des Requêtes simples « SELECT »
3. Distinguer les fonctions mono lignes/multilignes
4. Définir les Jointures, les Opérateurs ensemblistes et les sous-interrogations

1ère Partie :

Les Requêtes simples « la clause SELECT »

Objectifs de la partie :

Définir «LID», savoir sélectionner des colonnes avec ou sans doublons, avec ou sans des conditions connaître les Expressions arithmétiques, la Valeur « NULL », les Alias de colonne, les opérateurs logique et ceux de comparaison.

LID.

(1/2)

Le langage LID permet, comme son nom l'indique, d'interroger une BD. Il sert à **rechercher**, **extraire**, **trier**, mettre en forme des données et **calculer** d'autres données à partir des données existantes.

La structure de base d'une interrogation est formée des 3 clauses suivantes :

```
SELECT *|<liste champ(s)> FROM <nom_table>  
          WHERE <condition(s)> ;
```

LID.

(2/2)

- La clause SELECT : indique les colonnes à récupérer. (ou encore des champs projetés).
- La clause FROM indique le nom de la ou des table(s) impliquée(s) dans l'interrogation.
- La clause WHERE correspond aux conditions de sélection des champs.
- Chaque nom de champ ou de table est séparé par une virgule.

Sélection des colonnes. Toutes les colonnes

SELECT  Affiche tous les champs
de la table
FROM employees;

Résultat:

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID
100	Steven	King	SKING	 515.123.4567 	17/06/87	AD_PRES
101	Neena	Kochhar	NKOCHHAR	 515.123.4568 	21/09/89	AD_VP
102	Lex	De Haan	LDEHAAN	 515.123.4569 	13/01/93	AD_VP
103	Alexander	Hunold	AHUNOLD	590.423.4567	03/01/90	IT_PROG
104	Bruce	Ernst	BERNST	590.423.4568	21/05/91	IT_PROG
105	David	Austin	DAUSTIN	590.423.4569	25/06/97	IT_PROG

Sélection des colonnes. Une/plusieurs colonnes

```
SELECT first_name, last_name  
FROM employees;
```

Affiche les champs :
« last_name » et
« first_name »

Résultat:

FIRST_NAME	LAST_NAME
Ellen	Abel
Sundar	Ande
Mozhe	Atkinson
David	Austin
Hermann	Baer
Shelli	Baida
Amit	Banda
Elizabeth	Bates

Expressions arithmétiques.

Définition

- i. Une expression contient des données de type **NUMBER**, **DATE** et des **opérateurs arithmétiques**.

Opérateur	Description
+	Addition
-	Soustraction
*	Multiplication
/	Division

Expressions arithmétiques. Exemple 1

```
SELECT    min_salary + max_salary , min_salary - max_salary,
           min_salary * max_salary, min_salary / max_salary
FROM     jobs;
```

Résultat:

[illegible]

Expressions arithmétiques. Exemple 2

Ecrire la requête qui permet d'afficher le nom, prénom de tous les employés ainsi que leur salaire annuel.

FIRST_NAME	LAST_NAME	Salaire Annuel
Steven	King	288600
Neena	Kochhar	204600
Lex	De Haan	204600
Alexander	Hunold	108600
Bruce	Ernst	72600
David	Austin	58200
Valli	Pataballa	58200
Diana	Lorentz	51000

Valeur «NULL».

Définition

- NULL est une valeur qui n'est pas **disponible, non affectée ou inconnue.**
- NULL est différent de **zéro, espace ou chaîne vide**

Exemple 1:

```
SELECT employee_id, commission_pct  
FROM employees ;
```

Résultat:

EMPLOYEE_ID	COMMISSION_PCT
100	-
101	-
102	-
103	-
104	-
105	-

Valeur «NULL». Dans les expressions arithmétique.

Exemple 2 :

```
SELECT Employee_id, (1+commission_pct)*salary  
FROM employees;
```

Résultat:

EMPLOYEE_ID	(1+COMMISSION_PCT)*SALARY
100	-
101	-
102	-
103	-
104	-
105	-

Alias de colonne.

(1/2)

- Renomme un **en-tête** de colonne
- Suit le nom de colonne
- Peut utiliser le mot clé « **as** »
- Doit **obligatoirement** être inclus entre **guillemets** s'il contient des espaces, des caractères spéciaux ou s'il y a des majuscules et des minuscules

Alias de colonne. Exemple

(2/2)

```
SELECT first_name as nom, last_name prenom  
FROM employees;
```

Résultat:

NOM	PRENOM
Ellen	Abel
Sundar	Ande
Mozhe	Atkinson
David	Austin
Hermann	Baer

```
SELECT first_name as "nom de l'employee", last_name "prenom de l'employee"  
FROM employees;
```

Résultat:

Nom De L'employee	Prénom De L'employee
Ellen	Abel
Sundar	Ande
Mozhe	Atkinson
David	Austin
Hermann	Baer

Opérateur de concaténation.

- Concatène des colonnes **et/ou** des chaînes de caractères
- Est représenté par le symbole **||**

Exemple:

```
SELECT 'le nom est ' || first_name || ', le prénom est ' || last_name as "nom et  
    prenom de l'employe"  
FROM employees;
```

Résultat:

Nom Et Prénom De L'employé
le nom est Ellen, le prénom est Abel
le nom est Sundar, le prénom est Ande
le nom est Mozhe, le prénom est Atkinson
le nom est David, le prénom est Austin

Éliminer les doublons

- le mot-clé **DISTINCT** élimine les doublons.

Exemple:

```
SELECT DISTINCT(commission_pct)  
  
FROM employees;
```

Résultat:

COMMISSION_PCT
-
,15
,35
,4
,3
,2
,25
,1

Restriction avec la clause

« WHERE »

Syntaxe :

```
SELECT <nom_des colonnes> FROM <nom_de_la_table >  
WHERE <conditions>;
```

Exemple: liste des employés du département 20

```
SELECT last_name , first_name FROM employees  
WHERE department_id=20;
```

Résultat:

LAST_NAME	FIRST_NAME
Hartstein	Michael
Fay	Pat

2 lignes renvoyées en 0,05 secondes

Opérateurs de comparaison.

(1/2)

OPERATEUR	DESCRIPTION
=	Egal à
<	Inférieur à
<=	Inférieur à ou égal
>	Supérieur à
>=	Supérieur à ou égale à
<> Ou !=	Différent
BETWEEN val1 AND val2	val1 <= val <= val2
In (liste de valeurs)	Valeur de la liste
LIKE (_ :un caractère, %=plusieurs caractères)	Comme
IS NULL	Correspond à une valeur NULL

Opérateurs logiques.

(2/2)

OPERATEUR	DESCRIPTION
AND	Retourne TRUE si les deux conditions sont VRAIES
OR	Retourne TRUE si au moins une des conditions est VRAIE
NOT	Inverse la valeur de la condition TRUE si la condition est FAUSSE FALSE si la condition est VRAIE

Trier avec « ORDER by ».

Définition

Triez les lignes selon un ordre bien précis avec la clause ORDER BY:

- **ASC** : ordre croissant (par défaut)
- **DESC** : ordre décroissant

Remarque: La clause ORDER BY vient en dernier dans l'instruction SELECT

Trier avec « ORDER by ». Exemple

```
SELECT last_name, job_id, department_id, hire_date  
FROM employees  
ORDER BY hire_date DESC;
```

Résultat:

LAST_NAME	JOB_ID	DEPARTMENT_ID	HIRE_DATE
Kumar	SA_REP	80	21/04/00
Banda	SA_REP	80	21/04/00
Ande	SA_REP	80	24/03/00
Markle	ST_CLERK	50	08/03/00
Lee	SA_REP	80	23/02/00
Philtanker	ST_CLERK	50	06/02/00

Exercices d'applications.

(1/2)

(nous allons utilisé la table « employees » sous le schéma HR)

- Afficher la liste de tous les employés.
- Afficher le nom, le prénom, la date d'embauche de tous les employés et renommer les colonnes comme suit: « nom de l'employe », « prenom de l'employe » et « date dembauche »
- Afficher le nom et le prénom des employés triés par date d'embauche.
- Afficher les employés qui ont été embauché au cours de l'année 1990.
- Afficher la liste des employés dont le salaire est > 2800.
- Afficher la liste des employés des départements 10, 30 et 50.

Exercices d'applications.

(2/2)

- Afficher la liste des employés dont le salaire entre 4000 et 6000 du département 20
- Afficher la liste des employés dont la commission_pct est NULL.
- Afficher la liste des employés dont le nom commence par la lettre 'A'
- Afficher la liste des employés dont le prénom comporte la lettre 's'
- Afficher la liste des employés dont la deuxième lettre du nom est 'i'
- Afficher la liste des employés embauchés pendant les années entre 1986 et 1990
- Afficher la liste des employés dont le job est différent de 'CLERK' ou 'MANAGER'

2ème Partie : Les Fonctions Mono lignes

Objectifs de la partie:

Définir et savoir utiliser les :

- i. Fonctions de caractères
- ii. Fonctions de manipulation des caractères
- iii. Fonctions de manipulation des dates
- iv. Fonctions numériques
- v. Fonctions de conversions
- vi. Autres fonctions

Fonctions de caractères. Conversion Minuscule/Majuscule(1/2)

Il existe trois fonctions principales:

- **LOWER** : convertir les caractères « M » en « m »
- **UPPER** : convertir les caractères « m » en « M »
- **INITCAP** : convertir l'initiale de chaque mot en « M » et les caractères suivants en « m »
- **Remarque:** Une fonction **mono ligne** est une fonction qui s'applique enregistrement par enregistrement.

Fonctions de caractères. Conversion Minuscule/Majuscule(2/2)

SELECT LOWER(last_name) as "prénom" FROM employees;

Prénom
abel
ande

SELECT UPPER(last_name) as "prénom" FROM employees;

Prénom
ABEL
ANDE

SELECT INITCAP(last_name) as "prénom" FROM employees;

Prénom
Abel
Ande

Fonctions de manipulation des caractères. Définitions (1/2)

- **CONCAT**_(chaine1,chaine2) : concatène 2 chaînes = ||
- **SUBSTR**_(chaîne, pos, taille) : extrait une sous chaîne d'une autre chaîne
- **LENGTH**_(ch) : taille d'une chaîne en caractères
- **INSTR**_(ch,sch) : position d'une chaîne de caractères dans une autre chaîne
- **TRIM**_(ch) : élimine les espaces à gauche et à droite
- **LTRIM**_(ch) : élimine les espaces à gauche
- **RTRIM**_(ch) : élimine les espaces à droite

Fonctions de manipulation des caractères. Définitions (2/2)

- **LPAD** (chaîne, nbr, caract): complète une chaîne de caractères sur la gauche avec une autre chaîne pour avoir n caractères.
- **RPAD** (chaîne, nbr, caract): complète une chaîne de caractères sur la droite avec une autre chaîne pour avoir n caractères.
- **ASCII** (chaîne): retourne le code ascii du premier caractère de la chaîne.
- **CHR** (nbr): retourne le caractère (inverse de ascii).

Fonctions de manipulation des caractères. Exemples

```
SELECT  'station' ch1, 'agil' ch2,  
        Concat ('station 1','agil') "la station",  
        Substr ('station 1',9,1) "numéro station",  
        Length ('agil') "longueur ch1",  
        Instr ('agil','i') "position de i",  
        Length (Trim(' agil ')) "trim",  
        Length (Rtrim(' agil ')) "Rtrim",  
        Length (Ltrim(' agil ')) "Ltrim",  
        Rpad ('agil',8,'*') "RPAD", Lpad ('agil',8,'-') "LPAD",  
        ASCII ('test') " code ascii " , CHR(75)  
  
FROM  Dual;
```

Fonctions numériques. Présentation

- **ROUND** (x, n) : Arrondir la valeur de x à la précision spécifiée n
- **TRUNC** (x, n) : Tronquer la valeur de x à la précision spécifiée n
- **FLOOR** (x) : si $n < x < n+1$ alors $\text{FLOOR}(x) = n$
- **CEIL** (x) : si $n < x < n+1$ alors $\text{CEIL}(x) = n+1$
- **MOD** (x, y) : reste de la division de x sur y
- **REMAINDER** (m, n) : reste d'une division calculé comme suit :
 $m - (q * n)$ avec q égale à la partie entière du quotient

Fonctions numériques. Round

(1/2)

```
SELECT round(123.646, 0) "0 chiffres" , round(123.646, 1) "1 chiffres",  
        round(123.646, 2) "2 chiffres", round(123.646, 3) "3 chiffres"  
FROM Dual;
```

Résultat:

0 Chiffres	1 Chiffres	2 Chiffres	3 Chiffres
124	123.6	123.65	123.646

Fonctions numériques. Round

(2/2)

```
SELECT round(123.646, -1) "1 chiffres", round(123.646, -2) "2 chiffres",  
       round(123.646, -3) "3 chiffres"  
FROM Dual;
```

Résultat:

1 Chiffres	2 Chiffres	3 Chiffres
120	100	0

Fonctions numériques. TRUNC

```
SELECT      TRUNC (123.646, 0) "n=0", TRUNC (123.646, 1) "n=1",  
            TRUNC (123.646, 2) "n=2", TRUNC (123.646, 3) "n=3", TRUNC (123.636, -1)  
            "n=-1", TRUNC (123.646, -2) "n=-2", TRUNC (123.646, -3) "n=-3"  
FROM Dual;
```

Résultat:

N=0	N=1	N=2	N=3	N=-1	N=-2	N=-3
123	123.6	123.64	123.646	120	100	0

Fonctions numériques. FLOOR/ CEIL

```
SELECT  floor(-2.56), floor(-2.02), floor(-2.8),  
        ceil(-2.56), ceil(-2.02), ceil(-2.8)  
FROM dual;
```

Résultat:

FLOOR(-2.56)	FLOOR(-2.02)	FLOOR(-2.8)	CEIL(-2.56)	CEIL(-2.02)	CEIL(-2.8)
-3	-3	-3	-2	-2	-2

```
SELECT  floor(2.56), floor(2.02), floor(2.8),  
        ceil(2.56), ceil(2.02), ceil(2.8)  
FROM dual;
```

Résultat:

FLOOR(2.56)	FLOOR(2.02)	FLOOR(2.8)	CEIL(2.56)	CEIL(2.02)	CEIL(2.8)
2	2	2	3	3	3

Fonctions numériques. MOD/ REMAINDER

```
SELECT  MOD (12, 4), MOD (16, 6)
FROM Dual;
```

Résultat:

MOD(12,4)	MOD(16,6)
0	4

```
SELECT  remainder (15, 6), remainder (15, 5),
        remainder (15, 4), remainder (-15,4)
FROM Dual;
```

$15 - (2 \times 6) = \mathbf{3}$ / $15 - (3 \times 5) = \mathbf{0}$ / $15 - (4 \times 4) = \mathbf{-1}$ / $-15 - (-4 \times 4) = \mathbf{1}$

Résultat:

REMAINDER(15,6)	REMAINDER(15,5)	REMAINDER(15,4)	REMAINDER(-15,4)
3	0	-1	1

Fonctions de manipulation des dates.

(1/3)

- **MONTHS_BETWEEN (date1,date2):** nombre de mois entre deux dates
- **ADD_MONTHS (date, nb_mois):** Ajoute des mois calendaires à une date
- **NEXT_DAY (date, jour) :** le jour suivant
- **LAST_DAY (date) :** le dernier jour du mois
- **ROUND (date, précision) :** arrondi une date
- **TRUNC (date, précision) :** tronque une date
- **EXTRACT (day/month/year/hour/minute/seconde from date):** extraction du jour, mois, année, heure, minute et seconde

Fonctions de manipulation des dates.

(2/3)

```
SELECT  sysdate "date du jour",  
        sysdate+2 "Date jr plus 2jrs",  
        to_date ('10/09/2012','dd/mm/yyyy')-10 "date du jour moins 10" ,  
        Trunc (sysdate - to_date('01/05/2012','dd/mm/yyyy'),0)  
        "tronquer une date"  
  
FROM dual ;
```

Résultat:

Date Du Jour	Date Jr Plus 2jrs	Date Du Jour Moins 10	Tronquer Une Date
09-MAY-12	11-MAY-12	31-AUG-12	8

Fonctions de manipulation des dates.

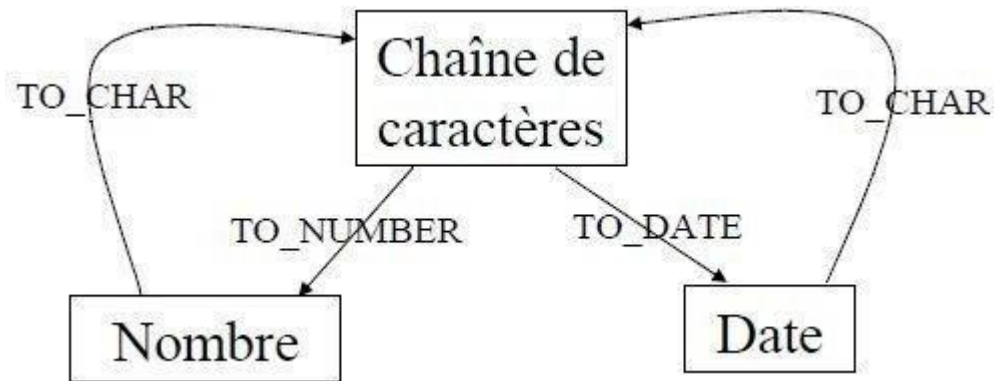
(3/3)

```
SELECT
Trunc (months_between (to_date ('2012/01/01', 'yyyy/mm/dd'), to_date
('2012/03/15', 'yyyy/mm/dd') ),2) "nbr de mois",
next_day(to_date ('2012/03/01', 'yyyy/mm/dd'), 'Monday') "lundi suivant",
last_day(to_date ('2012/03/01', 'yyyy/mm/dd')) "dernier jr du mois mars12" ,
add_months(to_date ('2012/03/01', 'yyyy/mm/dd'),3) "ajouter 3 mois"
FROM Dual;
```

Résultat:

Nbr De Mois	Lundi Suivant	Dernier Jr Du Mois Mars12	Ajouter 3 Mois
-2.45	05-MAR-12	31-MAR-12	01-JUN-12

Fonctions de conversions.



- **TO_DATE** : Convertir une Chaîne en format Date
- **TO_NUMBER** : Convertir une Chaîne en format Numérique
- **TO_CHAR** : Convertir une Date /un nombre en une Chaîne

Fonctions de conversions.

Options avec To_char

OPTION	DESCRIPTION
yyyy	l'année (quatre chiffres)
month	nom complet du mois en minuscule
mon	abréviation du nom du mois en minuscule (trois caractères)
day	nom complet du jour en minuscule
DD	jour du mois (01-31)
D	jour de la semaine (de 1 à 7, dimanche étant le 1)
dy	abréviation du nom du jour en minuscule (3 caractères)
ddd	jour de l'année (001-366)
q	trimestre
w	numéro de semaine du mois (de 1 à 5) (la première semaine commence le premier jour du mois.)
ww	numéro de semaine dans l'année (de 1 à 53) (la première semaine commence le premier jour de l'année.

Autres Fonctions.

(1/5)

- **NVL (nom_col, valeur) :** remplace une valeur nulle
- **NVL2 (expr, val1, val2) :** si expr n'est pas nulle alors elle est remplacée par val1 sinon par val2.
- **NULLIF (val1, val2) :** si val1= val2 la valeur NULL est retournée sinon val1
- **Case :** évalue une liste de conditions et retourne un résultat parmi les cas possibles

Autres Fonctions.

(2/5)

- Ecrire les requêtes **SELECT** qui permettent de:
 - Remplacer « `commission_pct` » par 0 au niveau de la table « `employees` » si elle est null
 - Remplacer « `commission_pct` » par 0 au niveau de la table « `employees` » si elle est null sinon par 1
 - Afficher la liste des employés en ajoutant une colonne « `nom_departement` » qui affiche :
 - Si `department_id=10` , `nom_departement='depart_10'`
 - Si `department_id=20` , `nom_departement='depart_20'`
 - Si `department_id=30` , `nom_departement='depart_30'`
 - ...

Autres Fonctions.

(3/5)

- **ROW_NUMBER** : retourne le numéro séquentiel d'une ligne d'une partition d'un ensemble de résultats, en commençant à 1 pour la première ligne de chaque partition. **Ne prend pas en considération les doublons**
- **RANK**: retourne le rang de chaque ligne au sein de la partition d'un ensemble de résultats. **Compte les doublons mais laisse des trous**
- **DENSE_RANK** : retourne le rang des lignes à l'intérieur de la partition d'un ensemble de résultats, sans aucun vide dans le classement.
Compte les doublons mais ne laisse pas des trous

Autres Fonctions.

(4/5)

```
SELECT
```

```
    Row_number() over (partition by <col x> order by <col_y>  
    DESC/ASC), Col1,...colN
```

```
FROM <nom_table> ;
```

A retenir:

- La clause **<order by>** est obligatoire.
- La clause **<partition by>** est facultative, est utilisée pour faire un ordre par ensemble de lignes selon **<colx>**.
- **Rank** et **dense_rank** ont la même syntaxe.

Autres Fonctions.

(5/5)

Ecrire les requêtes **SELECT** qui permettent de :

- Afficher la liste des employees numérotés par « department_id ».
- Afficher la liste des employees numérotés par département et selon le salaire **décroissant** :
 - Utiliser « row_number »
 - Utiliser « rank »
 - Utiliser « dense_rank »

3ème Partie :

Les Fonctions Multi lignes (ou fonctions de groupe)

Objectif du cours :

Définir la clause « GROUP BY »,
la clause « HAVING »

Fonctions Multi lignes. Définitions

(1/3)

Une fonction **mono ligne** est une fonction qui s'applique enregistrement par enregistrement.

Une fonction **multi ligne** ou fonction de groupe s'applique sur un groupe d'enregistrements et donne un résultat par groupe.

- Clause **GROUP BY** : Définit le critère de groupement pour la fonction
- Clause **HAVING** : Permet de mettre des conditions sur les groupes d'enregistrements

Fonctions Multi lignes. Définitions

(2/3)

- **COUNT** : Nombre de lignes,
- **SUM** : Somme des valeurs,
- **AVG** : Moyenne des valeurs,
- **MIN** : Minimum,
- **MAX** : Maximum,
- **STDDEV**: Ecart type,
- **VARIANCE**: Variance

Fonctions Multi lignes. Exemple1

```
SELECT      COUNT (salary) as count,  
            Trunc (SUM (salary),2) as sum,  
            MIN (salary) as min,  
            MAX (salary) as max,  
            Trunc (AVG (salary),2) as avg,  
            Trunc (VARIANCE (salary),2) as variance,  
            Trunc (STDDEV(salary),2) as ecart  
FROM employees;
```

Résultat:

COUNT	SUM	MIN	MAX	AVG	VARIANCE	ECART
107	691400	2100	24000	6461,682	15283140,539	3909,365

Fonctions Multi lignes. Syntaxe de la clause GROUP BY

SELECT <colonnes>, <fonction de groupe>

FROM table

WHERE <conditions>

GROUP BY <col> | <expr>

HAVING <conditions>

ORDER BY <col> | <expr>

Fonctions Multi lignes. Exemple2

```
SELECT department_id,  
COUNT(employee_id) "nbr employees"  
FROM employees  
  
GROUP BY department_id;
```

DEPARTMENT_ID	Nbr Employees
100	6
30	6
-	1
90	3
20	2
70	1
110	2
50	45
80	34
40	1
60	5
10	1

12
Départements

```
SELECT department_id, count(employee_id) "nbr employees"  
FROM employees  
  
GROUP BY department_id  
  
HAVING department_id < 40;
```

DEPARTMENT_ID	Nbr Employees
30	6
20	2
10	1

4ème Partie : Les Jointures

Objectifs de cette partie :

- i. Equijointure et jointure interne, Jointure naturelle
- ii. Jointure externe
- iii. Non équijointure
- iv. Produit cartésien

Les Jointures. Définition.

- Une jointure permet **d'extraire** des données de **plusieurs tables** à la fois.
- La condition de jointure peut être exprimée dans la clause « WHERE » ou « ON ».
- **Précédez** le nom de la colonne par le nom de la table lorsque celui-ci figure dans plusieurs tables

Types de Jointures.

- Equijointure ou jointure interne
- Jointure naturelle
- Jointure externe
- Non équijointure
- Produit cartésien

Equijointure. Définition

- Une jointure **interne** ou **équijointure** est une jointure avec une condition de jointure contenant **un opérateur d'égalité**.
- C'est la plus répandue, elle combine les lignes qui ont des valeurs **équivalentes** pour les colonnes de la jointure.
- Généralement dans ce type de jointure, ce sont les **Primary Key** et **Foreign Key** qui sont utilisées.

Equijointure. Syntaxe

(1/2)

```
SELECT table1.column, table2.column
```

```
FROM table1
```

```
INNER JOIN table2
```

```
ON (table1.column_name = table2.column_name);
```


Equijointure. Syntaxe

(2/2)

```
SELECT      T1.Colonne1, ...T1.ColonneN,  
            T2.Colonne1, ...T2.ColonneN  
FROM table1 T1 INNER JOIN table2 T2  
ON T1.C1=T2.C1  
WHERE <Condition(s)>
```

- ✓ **T1** : Alias de la table : table1
- ✓ **T2** : Alias de la table : table2
- ✓ **C1** : colonne permettant la jointure entre les 2 tables
(primary key + foreign key)

Équivalent à :

```
SELECT      T1.Colonne1, ...T1.ColonneN,  
            T2.Colonne1, ...T2.ColonneN  
FROM T1 ,T2  
WHERE T1.C1=T2.C1 AND <Condition(s)>
```

Equijointure. **Exemple**

Exemple1:

```
SELECT nomE, e.numdept, nomdept  
FROM emp e INNER JOIN dept d  
ON e.numdept=d.numdept;
```

Exemple2:

```
SELECT nomE, e.numdept, nomdept  
FROM emp e INNER JOIN dept d  
ON e.numdept=d.numdept  
WHERE e.numdept=11;
```

Jointure naturelle.

Définition

- La jointure naturelle est une **équijointure**.
- La clause **NATURAL JOIN** est basée sur toutes les colonnes des deux tables portant **le même nom**.
- Elle sélectionne les lignes des deux tables dont les valeurs sont **identiques** dans toutes les colonnes qui correspondent.
- Si les colonnes portant le même nom présentent des types de données **différents**, une erreur est renvoyée.

Jointure naturelle. Syntaxe

```
SELECT colonne1, colonne2...  
FROM Table1  
NATURAL JOIN Table2  
WHERE Condition(s) ;
```



Conditions autres que la condition
de jointure

Jointure naturelle. Exemple

Exemple 1:

```
SELECT nomE, numdept,nomdept  
FROM emp NATURAL JOIN dept;
```

Exemple 2:

```
SELECT nomE, numdept,nomdept  
FROM emp NATURAL JOIN dept  
WHERE numdept=11;
```

Non équijointure.

Définition

- Il s'agit là d'utiliser n'importe quelle condition de jointure entre deux tables, exceptée la stricte égalité. Ce peuvent être les conditions suivantes :

>	supérieur
>=	supérieur ou égal
<	inférieur
<=	inférieur ou égal
<>	différent de
IN	dans un ensemble
LIKE	correspondance partielle
BETWEEN ... AND ...	entre deux valeurs
EXISTS	dans une table

Jointure externe.

Définition

- JOINTURES EXTERNES = **OUTER JOINS**.
- Une jointure externe **élargie** le résultat d'une jointure interne (INNER JOINS) et permet d'extraire des enregistrements qui ne répondent pas aux critères de jointure.
- Une jointure externe renvoie **toutes les lignes** qui satisfont la condition de jointure et retourne également une partie de ces lignes de la table pour laquelle aucune des lignes de l'autre ne satisfait pas la condition de jointure.

Jointure externe. Types de jointures externes

```
SELECT t1.column, t2.column..  
FROM table1 t1  
  
LEFT | RIGHT | FULL OUTER JOIN table2 t2 ON  
(t1.column_name = t2.column_name);
```

- Le sens de la jointure externe **LEFT** ou **RIGHT** de la clause **OUTER JOIN** désigne la table **dominante**.
- **FULL = INNER + LEFT + RIGHT**

Jointure externe. Gauche

Exemple:

```
SELECT nomE, e.numdept, nomdept  
FROM dept d LEFT JOIN emp e  
ON e.numdept=d.numdept
```

NOME	NUMDEPT	NOMDEPT
afia tarek	11	informatique
benflen salah	11	informatique
bensalah salma	11	informatique
hamid ali	11	informatique
benafia ahmed	11	informatique
benflen ali	21	télécommunication
benafia tarek	21	télécommunication
hena salma	31	électromécanique
-	-	genie logiciel

Jointure externe. Droite

Exemple:

```
SELECT nomE, e.numdept, nomdept  
FROM dept d RIGHT JOIN emp e  
ON e.numdept=d.numdept
```

NOME	NUMDEPT	NOMDEPT
test test	-	-
benafia ahmed	11	informatique
hamid ali	11	informatique
bensalah salma	11	informatique
afia tarek	11	informatique
benflen salah	11	informatique
benflen ali	21	télécommunication
benaf salma	31	électromécanique
benafia tarek	21	télécommunication

Jointure externe. complète

Exemple:

```
SELECT nomE, e.numdept, nomdept  
FROM dept d FULL JOIN emp e  
ON e.numdept=d.numdept
```

NOME	NUMDEPT	NOMDEPT
afia terek	11	informatique
benflen salah	11	informatique
bensalah salma	11	informatique
hamid ali	11	informatique
benafia ahmed	11	informatique
benflen ali	21	télécommunication
benafia terek	21	télécommunication
bena salma	31	électromécanique
-	-	genie logiciel
test test	-	-

Produit Cartésien.

Définition

On obtient un produit cartésien lorsque :

- Une condition de jointure est omise
- Une condition de jointure est incorrecte
- A chaque ligne de la table 1 sont jointes toutes les lignes de la table 2.
- Le nombre de lignes renvoyés est égal $n1 * n2$ où
 - ✓ $n1$ est le nombre de lignes de la table 1
 - ✓ $n2$ est le nombre de lignes de la table 2

Produit Cartésien. Syntaxe

```
SELECT « nom_colonne1 », « nom_colonne2 »...  
FROM table1 t1  
CROSS JOIN table2 t2
```

Exemple:

```
Select * from employees CROSS JOIN departments
```

→ On va avoir **107*27** lignes = 2889 lignes

Produit Cartésien. Exemple

SELECT * From emp e

INNER JOIN dept d

ON e.numdept=e.numdept

▪ OU

SELECT * From emp, dept

→ On va avoir **9*4** lignes

Jointure. Récapitulatif

Jointure interne	<pre>SELECT FROM <table gauche> [INNER]JOIN <table droite> ON <condition de jointure></pre>
Jointure externe	<pre>SELECT FROM <table gauche> LEFT RIGHT FULL [OUTER]JOIN <table droite> ON <condition de jointure></pre>
Jointure naturelle	<pre>SELECT FROM <table gauche> NATURAL JOIN <table droite> (USING <noms de colonnes>]</pre>
Jointure croisée	<pre>SELECT FROM <table gauche> CROSS JOIN <table droite></pre>

5ème Partie : Les Opérateurs ensemblistes

- i. UNION
- ii. UNION ALL
- iii. INTERSECT
- iv. MINUS

Les opérateurs ensemblistes.

OPERATEUR	DESCRIPTION
INTERSECT	Ramène toutes les lignes communes aux deux requêtes
UNION	Toutes les lignes distinctes ramenées par les deux requêtes
UNION ALL	Toutes les lignes ramenées par les deux requêtes y compris les doublons
MINUS	Toutes les lignes ramenées par la première requête sauf les lignes ramenées par la seconde requête

Union(ALL). Exemple

- Créer une table d'archivage qui contient les informations des employés qui ont été embauchés après l'année 2009.
- Ecrire une requête qui va afficher tous les employés:

```
SELECT * from emp
```

```
Union (ALL)
```

```
SELECT * from emp2
```

Intersect/Minus. Exemple

Ecrire les requêtes permettant de :

- Afficher les employés qui **ont été archivés**.
- Afficher les employés qui **n'ont pas été archivés**.

```
SELECT * from emp
```

Intersect/minus

```
SELECT * from emp2;
```

Exercice d'application.

- Ecrire une requête qui permet d'afficher ce résultat à partir de la table 'dept' :

NOMDEPT	NUMDEPT
électromécanique	31
genie logiciel	41
informatique	11
télécommunication	21
TOTAL DES DEPARTEMENTS	4

6ème Partie :

Les Sous interrogations

- i. Sous interrogations mono lignes
- ii. Sous interrogations multi lignes

Sous interrogations. Définition

- La sous-interrogation (**requête interne**) est exécutée une **seule** fois avant la requête principale
- Le résultat de la sous-interrogation est utilisé par la requête principale (**requête externe**)
- Une sous-interrogation est utilisée dans les clauses suivantes :
 - **WHERE**
 - **HAVING**
 - **FROM**

Sous interrogations mono lignes.

Définition

Une sous-interrogation peut ramener **un seul résultat** on l'appelle sous interrogation mono ligne.

Les opérateurs de comparaison **mono ligne** sont :

=, !=, <, >, <=, >=

Sous interrogations mono lignes.

Exemple1

- **Exemple de Mise à jour avec une sous interrogation :**

l'employé numéro **112** est affecté au même poste et même département que l'employé numéro **111**

```
UPDATE employee_id SET (job_id, department_id)
= ( SELECT job_id, department_id
FROM employees
WHERE employee_id=111 )
WHERE employee_id=112 ;
```


Sous interrogations mono lignes.

Exemple2

Exemple de Suppression avec une sous interrogation :

Supprimer tous les employés du département 'SALES'

```
DELETE FROM employees  
WHERE Department_id =  
( SELECT department_id  
FROM departments  
WHERE department_name='SALES' );
```

Sous interrogations multi lignes.

(1/3)

Définition

- Une sous-interrogation peut ramener **plusieurs** lignes à condition que **l'opérateur de comparaison** admette à sa droite un **ensemble** de valeurs.
- Les opérateurs permettant de comparer une valeur à un ensemble de valeurs sont :
 - l'opérateur **IN** et
 - les opérateurs obtenus en ajoutant **ANY** ou **ALL** à la suite des opérateurs de comparaison classique :
=, !=, <, >, <=, >=.

Sous interrogations multi lignes.

Définition

(2/3)

- **ANY** : la comparaison sera vraie si elle est vraie pour au moins un élément de l'ensemble (elle est donc fausse si l'ensemble est vide).
- **ALL** : la comparaison sera vraie si elle est vraie pour tous les éléments de l'ensemble (elle est vraie si l'ensemble est vide).
- L'opérateur **IN** est équivalent à **= ANY**,
- L'opérateur **NOT IN** est équivalent à **!= ALL**.

Sous interrogations multi lignes. Définition (3/3)

Opérateur de comparaison	• Résultat
WHERE salaire IN (1200,1600,2000,2900)	•SALAIRE doit être égale à une des valeurs dans la liste
WHERE salaire NOT IN (1200,1600,2000,2900)	• SALAIRE ne doit pas être égale à une des valeurs dans la liste
WHERE salaire >ANY (1200,1600,2000,2900)	•SALAIRE doit être supérieur à au moins une des valeurs. Donc plus que le minimum (+ de 1200).
WHERE salaire <ANY (1200,1600,2000,2900)	•SALAIRE doit être inférieur à au moins une des valeurs. Donc moins que le maximum (- de 2900).
WHERE salaire >ALL (1200,1600,2000,2900)	•SALAIRE doit être supérieur au maximum de toutes les valeurs. Donc plus que le maximum (+ de 2900).
WHERE salaire <ALL (1200,1600,2000,2900)	•SALAIRE doit être inférieur au minimum de toutes les valeurs. Donc moins que le minimum (- de 1200).